Reuse semantics instead of repeating transport

Further simplifications of ProveIt's Rocq and Lean proofs
22 October 2026

Two recommendations. For Lean, derive quotient Replacement from the already-proved quotient collection theorem, using native equality. For Rocq, transport the common macro library across a closed unchanged subgraph instead of maintaining a renamed copy of its proofs. Neither proposal changes the machine, the calculus or the consistency target.

Scope. This review is pinned to main commit `6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`. It inspects actual source and upstream validation records. Our new proof drafts are *uncompiled*. No Lean, Rocq or Coq executable was available in this review environment. Source audits and finite mathematical checks below are not substitutes for kernel checking. No repository files were modified or committed.

1. What has changed since the previous review

The direct-seed proposal now has upstream component evidence: it returns `FullSeed mem` on the original domain, avoiding the first indiscernibility quotient. Its records report 116 fresh Lean compilations and 26 axiom reports, including generic `KEndpoint`, but explicitly not the literal machine endpoint. Its selected closure drops 179 to 178 modules and 7,534 bytes, while preserved legacy APIs increase touched production source by 6,795 bytes. Our earlier delivered proposal remains uncompiled; those separate upstream checks do not retroactively validate it.

The maintained phase-fourteen checkpoint adopts derived-rule and transitive-container sharing. Later quotient and dense-bridge changes were retained as proposals by merge `6901b164`; their presence in history does not establish maintained adoption. The new `eb931b07` quotient-unification proposal makes the QHyp development the single bisimulation development and supplies a CP/GIC adapter. It reports 601 fewer lines across fourteen files and checks of 25 reverse-closure modules, but excludes machine-specific endpoints and the facade. This is a strong existing proposal, not a new contribution of this report.

At our cutoff, K217 also reuses the checked macro-word template for AX and instantiates generalized boundary completeness at the seed. The latest commit removes a net 94 lines from `EnumDom`, `SeedDom` and `L2RootDom`, and retires the enumerator fork generator. Its author reports component builds, not a rebuilt `KEndpoint217`. These improvements should not be proposed again as unfinished work.

Batch 215 already contains a shared scheduler. It explicitly exposes the cost of abstraction: K215's scheduler alone grows 95 Lean lines, and the Rocq scheduler refactor grows 101 lines. Sharing induction can simplify maintenance without shortening a single endpoint. We therefore target *remaining duplicated semantic arguments*, rather than another generic scheduler interface.

2. Lean: obtain Replacement after entering the quotient

`KQ1Collect.quotient_collection` already establishes collection for every actual formula and every quotient valuation, with native equality. Nevertheless, `KQ1Replacement` constructs another raw equality-free matrix, proves bisimilarity invariance, transfers functionality to raw representatives, obtains interpreted Replacement, and descends its witness back to the quotient.

That file is 143 lines both in the pinned source and the quotient-unification overlay. Its raw proof path occupies lines 32–135. This is an opportunity to change the *order of reasoning*: transport collection once, then prove its consequences in the quotient. It is not a claim that all 143 lines can immediately be deleted.

Proposition 1 (Guarded collection gives a partial functional image)**.** *Let m be any binary relation on a type V, $a, b \in V$, and $R \subseteq V^2$. Suppose*

$$\begin{aligned} m(y,b) &\Longrightarrow \exists x\,[m(x,a) \wedge R(x,y)],\\ m(x,a) \wedge \exists y\, R(x,y) &\Longrightarrow \exists z\,[m(z,b) \wedge R(x,z)],\\ m(x,a) \wedge R(x,y) \wedge R(x,z) &\Longrightarrow y = z. \end{aligned}$$

Then $m(y,b) \leftrightarrow \exists x\,[m(x,a) \wedge R(x,y)]$ for every y.

Proof. The forward implication is support. For the reverse, take x with $m(x,a)$ and $R(x,y)$. Coverage supplies z in b with $R(x,z)$. Functionality gives $y = z$, so y is in b. □

The lemma requires neither totality of R, extensionality of m, nor an inhabitedness assumption. It uses literal equality, not bisimilarity. The supplied `GuardedImage.lean` and `GuardedImage.v` are self-contained proof drafts of this elementary statement, explicitly not compiled or integrated.

One formula adapter. The repository uses output at slot 0 and input at slot 1 for `relOf`. For the quotient membership relation m, set

$$R(x,y) := \mathrm{relOf}(m, \varphi, e)(y, x), \qquad \psi := \mathrm{Mem}(0,2) \wedge \mathrm{rename}(\rho, \varphi),$$

where $\rho(0) = 1$, $\rho(1) = 0$, and $\rho(k+2) = k+3$. Under environment (x, y, a, e), ψ means exactly $m(x,a) \wedge R(x,y)$. Use `Sat_rename_relOf` from `FirstOrder/Fol.lean` and `collectionInstance_congr` for relation transport; the valuation equality has only the cases 0, 1, and $k+2$. Crucially, rename φ itself, *not* its equality-free translation: we are already in the quotient.

Instantiate `quotient_collection` with this guarded formula and environment (a, e). Its support and coverage give the first two premises above. Existing `Functional (relOf ...)` supplies the third, even though the lemma only needs uniqueness on the domain a. The resulting witness proves the current `quotient_replacement` statement; `bridge_Repl` then preserves `quotient_repl_form` unchanged.

3. Lean dependencies, compatibility and a smaller endpoint interface

The import order is acyclic:

$$\text{Replacement} \to \text{Collect} \to \text{Power} \to \text{Separation}.$$

The collection proof currently uses separation's valuation-lifting API. Do not simultaneously derive Separation from collection without moving that infrastructure and rechecking dependencies. Replacement alone is the low-risk first change.

Honest size accounting. The quotient-unification proposal preserves old raw helper APIs through wrappers in `QuotientReplacement`. Adding the native proof while keeping every raw helper can increase total source size. First inventory references to `replacement_instance`, `interpreted_replacement`, `quotient_relOf` and the raw choice-form lemmas. Keep the public quotient theorems unchanged. A safe first layout is a lightweight `KQ1ReplacementCore` and CP/GIC `QuotientReplacementCore`, with old modules importing these cores and retaining raw helper APIs. Both `KQ1ZF` and `InductiveAttempts` must import the cores: the latter uses the quotient theorem and would otherwise retain the heavy path. Remove unused helpers only after a complete consumer and documentation inventory. Report whole maintained source, selected endpoint source and compatibility source separately. No measured net saving is claimed here.

Why the guard matters. Collection's support only promises that a selected output has *some* input; it does not automatically put that input in a. The membership conjunct in ψ is essential. Nor may functionality be dropped: collection can select one of two possible outputs rather than their full image. The enclosed finite sanity check exhausts 33,033 relation/domain/witness triples over domains of size 0–3, including 1,782 satisfying premise instances, and checks both negative controls. This supports the elementary argument, not the syntax adapter or a formal endpoint.

Secondary interface refinement. The all-leaf decoder-residual proposal now reports coverage of 4,751 leaves and 6,176 guarded outcomes with 51 pinned component modules. It supplies `source_halts_iff` for related source/tape states. Instead of making generic `KEndpoint` understand another coverage certificate, its smallest core need only consume

$$\text{SourceHalts}(p, ([], R_0)) \iff \text{TapeHalts}(M, T).$$

Together with the existing dense/sparse correspondence, blank-to-T boot run, source/calculus equivalence and logical equivalence, this yields the same endpoint by transitivity. Existing Boolean coverage and new residual coverage become alternative producers of that single premise. Keep the old API as a wrapper.

This is a refinement of the residual packet's already-proposed direct composition, not a new simulation theorem. The current endpoint is already short; the benefit is separation of certificate machinery from logical composition and independent validation of each producer, not a claimed line-count breakthrough. Boot forward preservation still needs the existing deterministic-halting argument. Positive progress in nonhalting simulation steps must remain inside each simulation provider.

4. Rocq: transport a closed macro subgraph once

The source review identifies `ProgLibPerm.v` as a renamed copy of `ProgLib219.v`: the 753-line suffix beginning with the determinism section is text-identical after replacing `dagPerm` by `dag219`. Prove once that equal node rows on a successor-closed set preserve the existing big-step `RM.run` relation, with its exact exit and final registers. Then replace the duplicated numerical macro proofs with checked transport wrappers. Generic state, node and flattening machinery should be shared separately; program-dependent flattening must still use the actual program.

5. Closed-subgraph transport: proof and acceptance plan

Let F be a set of node addresses and define $\mathrm{inside}_F(\texttt{ENode}\ n)$ as $n \in F$, with every primitive edge inside. Require, for every $n \in F$, equality $\texttt{nodes}\ P\ n = \texttt{nodes}\ Q\ n$ and that both edges of this pair are inside F. This gives

$$\mathrm{inside}_F(e) \implies \big(\texttt{RM.run}\ P\ e\ R\ x\ R' \iff \texttt{RM.run}\ Q\ e\ R\ x\ R'\big).$$

Induct on the finite `RM.run` derivation, not on fuel or DAG addresses. Primitive constructors are unchanged. In a node constructor, rewrite the equal pair and apply the induction hypotheses using closure of both children. Crucially, `run_node_cdr_continue1` also recursively runs the same node; its third induction hypothesis uses the original $n \in F$ premise. This handles loops without a separate termination argument. The reverse implication uses the same agreement and closure symmetrically.

All exits are preserved: `XNormal`, `XBreak k`, `XContinue k` and `XHalt`. In particular, `run_halt` says $\texttt{run}\ P\ \texttt{EHalt}\ R\ \texttt{XHalt}\ R$; it is not an existential-fuel or small-step stopping convention. The existing `sem` contract quantifies over initial registers and provides $\exists R',\ \texttt{run}\ P\ e\ R\ x\ R' \wedge \texttt{approx}\ R'\ m'$. Transfer its run witness and retain the same final-register approximation. First share the `St`/`approx` definitions so copied record types do not obstruct this adapter.

The source audit found 53 literal entry addresses in the library's semantic contracts. Their transitive footprint has 89 rows, all identical in `Dag219` and `DagPerm`; changed old rows 139, 141, 147 and 236 are outside it. The new dispatch/AX proofs remain variant-specific. These are source facts, not compiled transport certificates. Exact source pins and reproducible audit details accompany the report.

Implementation sequence.

1. Port the already-maintained Lean `RMTransfer` architecture, used by `L2Perm` and `L2Dom` to reuse macros. The inspected Rocq `RMEquiv` relates big-step and small-step semantics; `Bisim230.check_iff` transfers whole-program halting from a certificate. Neither supplies this arbitrary-edge, exit-and-register-preserving contract. The archived `k215-simplification-3/rocq/CFG_REPORT.md` already suggests this port for changed-increment CFGs. The new contribution here is the concrete 753-line duplication and its 89-row sharing boundary, not the general port idea.
2. Prove or reflect one footprint certificate containing entry inclusion, equal lookups and closure. A Boolean audit outside the kernel is not sufficient by itself.
3. Move the common macro proofs behind a program parameter and the footprint certificate, or preserve the old library and expose thin transferred lemmas for the new program. Preserve consumed theorem names and statements.
4. Update any producer that would regenerate the duplicate file. Rebuild both macro clients, variant dispatch/AX proofs and every affected source-to-machine theorem before claiming adoption.

Formal validation boundary. For Lean, check the native-image lemma, formula adapter, both quotient Replacement endpoints, the CP/GIC compatibility lane and then literal `KEndpoint215`. For Rocq, check the generic run-transport theorem, finite certificate and downstream endpoint under the production-designated toolchain, with recursive kernel checking. Upstream Coq 8.18 scheduler checks do not certify production Rocq 9.1 sources. Preserve axiom reports and exact transitive source bindings; distinguish reused objects from fresh compilation.

Recommendation order. First prototype the short native-image lemma and guarded adapter in Lean, and the closed-footprint run theorem in Rocq. Measure the resulting maintained proof source *including* adapters and certificates. Adopt only after complete client checking. The larger direct-seed, quotient-unification and residual-runtime proposals remain valuable, but their separate component receipts should not be combined into an unperformed full-machine validation.

Sources and reproducibility

Pinned source paths and hashes appear in `SOURCE_AUDIT.json`. The principal upstream records are the review cutoff, quotient-unification proposal, and batch 215's candidate-scheduling and decoder-residual proposal directories. The report's new arguments are mathematical proposals; no smaller machine, compiled proof reduction, proof-term size reduction or speedup is asserted.